REST API Tasarımı ve Belgelendirme

API Kaynak Tasarımı ve Veri Modelleri

Bu aşamada, ürün öneri sisteminin dış dünyaya nasıl açılacağını, hangi endpoint yapılarının kullanılacağını ve JSON formatlarını belirledim.

API Uç Noktaları (Endpoints)

Öneri sistemini hem kullanıcı bazlı hem de ürün bazlı çalışacak şekilde tasarladım:

Metot	Uç Nokta	Açıklama
GET	`/api/v1/recommendations/user/{userId}`	Kullanıcının geçmişine göre kişiselleştirilmiş ürünler getirir.
GET	`/api/v1/recommendations/product/{productId}`	Bakılan ürüne benzer (tamamlayıcı) ürünleri getirir.
POST	`/api/v1/recommendations/feedback`	Önerilen ürünün tıklanma/beğenilme durumunu algoritmaya geri bildirir.

İstek ve Cevap Formatı (Örnek)

İstek: `GET /api/v1/recommendations/user/4521?limit=5`

Cevap (200 OK):

```
{
  "status": "success",
  "data": {
    "userId": 4521,
    "recommendationType": "personalized",
    "items": [
      {
        "productId": "P-102",
        "name": "Kablosuz Kulaklık",
        "score": 0.98,
```

```
        "price": 1250.00
      },
      {
        "productId": "P-205",
        "name": "Bluetooth Mouse",
        "score": 0.85,
        "price": 450.00
      }
    ]
  },
  "metadata": {
    "processingTimeMs": 45
  }
}
```

Swagger / OpenAPI Spesifikasyonu (Özet)

Bu tasarımın belgelendirmesi için kullanılan temel yapı:

- **OpenAPI Version:** 3.0.0
- **Content-Type:** `application/json`
- **Hata Yönetimi:** Standart `400 Bad Request, 401 Unauthorized ve 404 Not Found` kodları tanımlanmıştır.

Güvenlik ve Erişim Kontrolü Tasarımı

Projemizin güvenliğini sağlamak ve API kaynaklarını yetkisiz erişimlerden korumak için sektörde yaygın olarak kullanılan standartları temel alan bir yapı kurguladım. Bu bölümde kimlik doğrulama, yetkilendirme ve sistemin kötüye kullanımını engelleyen mekanizmalar yer almaktadır.

1. Kimlik Doğrulama: JWT (JSON Web Token)

API erişimlerinde her seferinde kullanıcı adı ve şifre sormak yerine, daha güvenli ve performanslı olan **JWT** yapısını kullandım.

- **Akış:** Kullanıcı `/login` endpoint'ine geçerli bilgilerle istek attığında, sunucu ona dijital imzalı bir token döndürür.
- **Kullanım:** İstemci, bu token'ı sonraki isteklerinde HTTP Header kısmında `Authorization: Bearer <token>` şeklinde gönderir.

- **Güvenlik:** Şifreler veritabanında asla açık metin olarak tutulmayacak, **bcrypt** algoritması ile hashlenerek saklanacaktır.

Token süresini (TTL) kısa tutarak, çalınma durumunda riskin azaltılmasını hedefledim.

2. Yetkilendirme ve Rol Yönetimi (RBAC)

Sistemde herkesin her işleme erişmesini engellemek için "Rol Tabanlı Erişim Kontrolü" tasarladım. Temelde üç ana rol belirledim:

- **Admin:** Sistem ayarları ve tüm veri yönetimi (Silme/Güncelleme).
- **Kullanıcı:** Kendi profilini yönetme ve öneri alma.
- **Satıcı:** Ürün listeleme ve stok yönetimi.

Örneğin, bir ürünün sistemden tamamen silinmesi `(DELETE /api/v1/products/{id})` sadece **Admin** yetkisine sahip tokenlar ile yapılabilecek şekilde kısıtlanmıştır.

3. API Key ve Dış Servis Entegrasyonu

Sadece kullanıcılar değil, ileride sisteme dahil olabilecek üçüncü parti mikroservisler için de **API Key** yapısını ekledim. Bu anahtarlar `x-api-key` header'ı üzerinden kontrol edilecek ve gerekirse yönetim panelinden anlık olarak iptal edilebilecektir.

4. İstek Sınırlama (Rate Limiting)

API'nin bot saldırılarına veya yanlışlıkla oluşabilecek sonsuz döngülere karşı çökmesini engellemek için basit bir hız sınırlayıcı planladım:

- **Giriş Denemeleri:** Dakikada en fazla 5 istek (Brute-force koruması).
- **Öneri API:** Dakikada 30 istek.
- **Genel Sınır:** Kullanıcı başına dakikada 100 istek.

Limit aşıldığında sistem standart `429 Too Many Requests` hatasını döndürerek sunucuyu korumaya alacaktır.

Performans ve Ölçeklenebilirlik Tasarımı

Öneri algoritmaları işlemciyi ve veritabanını yoğun kullanan yapılar olduğu için, sistemin yüksek trafikte tıkanmaması adına üç ana strateji belirledim:

1. Redis ile Önbellekleme (Caching)

Öneri algoritması her çalıştığında arka planda karmaşık hesaplamalar döner ve bu işlem 100-200ms sürebilir. Kullanıcı sayfayı her yenilediğinde bu maliyete katlanmamak için **Redis** katmanını devreye aldım.

- **Cache HIT/MISS:** Sistem önce Redis'e bakar; veri varsa **~5ms** içinde döner (HIT). Yoksa hesaplama yapılır ve Redis'e kaydedilir (MISS).
- **Akıllı Anahtar (Key) Tasarımı:** Verilerin tazeliğini korumak için farklı yaşam süreleri (TTL) belirledim. Kullanıcı davranışları hızlı değiştiği için kişisel önerileri 5 dakika, daha genel olan kategori önerilerini ise 1 saat önbellekte tutuyoruz.

2. Veritabanı Yükünü Azaltma Teknikleri

Önbelleğin yetmediği durumlarda veritabanı darboğazını engellemek için şu dört yöntemi uyguladım:

- **Okuma Replikası (Read Replica):** Yazma işlemlerini ana sunucuda (Master), okuma işlemlerini ise kopya sunucularda (Replica) yaparak ana sunucu yükünü **%80** oranında azalttım.
- **Bağlantı Havuzu (Connection Pooling):** Her istekte yeni DB bağlantısı açma maliyetini sıfırlamak için **PgBouncer** yapısını kurguladım.
- **Asenkron Feedback Yönetimi:** Kullanıcı tıklamalarını veritabanına anlık yazmak yerine bir mesaj kuyruğuna (RabbitMQ gibi) atarak sistem yanıt süresini **10ms** seviyesine çektim.
- **İndeksleme:** Sık sorgulanan alanlara (userId vb.) B-tree indeksleri ekleyerek sorgu hızını 10 kat artırdım.

3. Yük Dengeleme (Load Balancing) ve Ölçeklenebilirlik

Sistemin yatayda büyüyebilmesi ve tek bir sunucuya bağımlı kalmaması için şu kararları aldım:

- **Round-Robin Algoritması:** Gelen istekleri sunuculara sırayla dağıtarak dengeli bir yük paylaşımı sağladım.

- **Stateless (Durumsuz) Mimari:** Sunucuların birbirinden bağımsız çalışabilmesi için kullanıcı oturum verilerini sunucu üzerinde değil, merkezi bir Redis üzerinde tuttum.
- **Otomatik Hata Toleransı (Failover):** Yük dengeleyici her 10 saniyede bir sunuculara sağlık kontrolü (Health Check) yapar. Arızalı bir sunucu tespit edilirse sistem otomatik olarak devre dışı bırakılır ve kullanıcı bunu hissetmez.

Performans Hedefleri Özeti

Strateji	Kullanılan Teknik	Kazanım
Önbellekleme	Redis & TTL Yönetimi	Yanıt süresi: 150ms → **5ms**
DB Optimizasyonu	Read Replica & İndeksleme	Master yükünde **%80** düşüş
Hız	Asenkron Yazma (Kuyruk)	Feedback yanıtı: **10ms**
Ölçeklenebilirlik	Load Balancer	Binlerce eşzamanlı istek kapasitesi

Sonuç olarak; bu tasarım sayesinde sistem sadece bugünkü trafiği değil, kullanıcı sayısı 10 kat artsa bile sadece yeni sunucular eklenerek karşılanabilecek bir esnekliğe kavuşmuştur.